۱. معرفی کلی

هدف اصلی این اسکریپت، تشخیص تصاویر تکراری در میان پرونده‌های درمانی با استفاده از تکنیک‌های چندگانه و بهینه‌سازی پردازش (از جمله پردازش موازی) می‌باشد. این مدل علاوه بر شناسایی دقیق تصاویر کاملاً یکسان (با هش فایل) قابلیت تشخیص شباهت‌های بصری بین تصاویر (با هشی پیکسلی) را نیز داراست. نتایج نهایی به صورت یک فایل اکسل تولید شده و شامل گزارش‌های جامع از حالت‌های مختلف (تکراری‌های دقیق، موارد مشکوک، آمار پردازش و خطاها) می‌باشد.

ورودی و تنظیمات پیکربندی

کد اصلی با استفاده از چندین متغیر کلیدی، محیط اجرای خود را تنظیم کرده و فایل‌های ورودی/خروجی مهم را تعریف می‌کند. در این بخش به تفصیل به این متغیرها اشاره می‌کنیم:

تنظیمات مسیرها و فایل‌های مرجع

DIR _\DIRBASE :مسیر اصلی داده‌ها و پرونده‌ها که تمامی عملیات خواندن و پردازش روی این دایرکتوری انجام می‌شود.

PATHJSON_JSON :مسیر فایل JSON شامل لیست ادعاها یا پرونده‌های مورد بررسی را مشخص می‌کند. این فایل شامل اطلاعات و پارامترهای مربوط به پرونده (مانند نام پرونده، مسیر فایل، شناسه پرونده) است که برای شناسایی مجموعه‌ای از پرونده‌ها به منظور بررسی تصاویر تکراری استفاده می‌شود

OUT_EXCELOUT :آرشیو اطلاعاتی پایه فراهم می‌کند تا از آن برای اخذ لیست تصاویر جهت پردازش استفاده گردد

مسیری که خروجی نهایی کد به صورت یک فایل اکسل ذخیره خواهد شد. در این فایل، نتایج مختلف مانند جفت پرونده‌های دارای تصاویر تکراری، پرونده‌های مشکوک، آمار کلی پردازش و لیست خطاها درج می‌شود. برای مثال:

D\:fruad−ftp\exactallduplicatepairs

FILECACHE_CACHE: این فایل به منظور ذخیره‌سازی هش‌های تصاویر پردازش شده استفاده می‌شود تا از پردازش مجدد و تکراری جلوگیری شود. ذخیره قابل نگهداری نتیجه‌های محاسبه شده به‌خصوص در محیط‌های با حجم بالای داده، باعث افزایش سرعت و کاهش مصرف منابع می‌شود.

---

۲. مراحل پردازش و منطق اجرایی

قسمت اصلی کد شامل مراحل گوناگونی است که هر کدام با هدف دسته‌بندی تصاویر بر اساس شناسایی تکراری‌ها و بهبود سرعت و دقت پردازش طراحی شده‌اند:

۲.۱. بارگذاری داده‌ها

- ابتدا داده‌های مورد نیاز از فایل JSON (PATH_PATHJSON) خوانده می‌شود.

- اطلاعات موجود در فایل JSON شامل لیست پرونده‌های درمانی، مسیر هر پرونده، و احتمالا اطلاعات متادیتا از قبیل شناسه پرونده، تاریخ و غیره می‌باشند.
- این اطلاعات به عنوان ورودی اولیه برای فرآیند پردازش تصاویر به کار گرفته می‌شوند.

۲.۲. استخراج هش‌ها (Hash Extraction)

این بخش به دو روش برای شناسایی تکراری بودن تصاویر می‌پردازد:

الف) file_fileexact

در این روش، ابتدا هش SHA۲۵٦ کل فایل محاسبه می‌شود.

ب) pixel_ pixelexact

- در صورتی که تصاویر ممکن است از نظر فرمت، اندازه یا متادیتا تغییراتی داشته باشند اما محتوای بصری آن‌ها یکسان باقی بماند، از روش استخراج هش پیکسلی استفاده می‌شود.
- پس از تغییر اندازه، هش بر مبنای پیکسل‌های تصویر محاسبه می‌شود که امکان تشخیص تصاویر مشابه با تغییر فرمت یا متادیتا را فراهم می‌سازد.

۲.۳. فیلترهای پیش‌پردازش

برای بهبود کارایی و جلوگیری از پردازش‌های غیرضروری، دو نوع فیلتر پیش‌پردازشی اعمال می‌شود: صفحه سفید و تصاویر رادیولوژی

۲.۴. پردازش موازی (Parallel Processing)

- استفاده از کتابخانه ProcessPoolExecutor برای پردازش همزمان چندین تصویر.
- تقسیم‌بندی وظایف پردازشی به چندین فرایند مجزا
- در صورتی که سیستم GPU و کتابخانه‌های مرتبط مانند torch در دسترس باشد، امکان استفاده از شتاب‌دهنده‌های سخت‌افزاری نیز فراهم شده است.

---

۳. ملاحظات تکمیلی

علاوه بر تنظیمات ورودی و پردازش موازی، چند نکته مهم در طراحی این سیستم حاضر است:

- قابلیت مقیاس‌پذیری:
  - استفاده از کش فایل (FILECACHE_CACHE) به‌طور قابل اعتماد از پردازش‌های تکراری جلوگیری می‌کند.

- قابلیت اطمینان و ثبت خطاها:
  - تمام خطاهای احتمالی در فرایند پردازش ثبت و گزارش داده می‌شوند
- انعطاف‌پذیری ورودی:
  - تنظیمات فایل JSON ضمن فراهم کردن یک منبع مرکزی جهت مدیریت پرونده‌ها، امکان تغییر مقیاس داده در سیستم را تضمین می‌کند.

---

۴. نقش کش در این کد

در این اسکریپت، کش برای ذخیره نتایج محاسبه هش تصاویر استفاده می‌شود تا:

- از محاسبه دوباره SHA۲۵۶ فایل‌ها و هش پیکسلی جلوگیری شود
- زمان پردازش به‌شدت کاهش یابد
- امکان **Resume** پردازش پس از توقف (Crash / Restart) فراهم شود
- کش به‌صورت یک فایل pickle روی دیسک ذخیره می‌شود

ساختار کلی کش:

- یک دیکشنری پایتونی
- کلیدها معمولاً شامل اطلاعات تصویر (مثلاً مسیر فایل)
- مقادیر شامل هش‌ها و متادیتاهای مرتبط با آن تصویر

۱) در کد دو تابع اصلی برای مدیریت کش وجود دارد:

۲) load_cache(cache_path:str,fingerprint:str)→dict|None

۳) save_cache(cache_path:str,fingerprint:str,data:dict)→ None

تابع save_cache با دریافت مسیر فایل کش(cache_pathcache) ، اثرانگشت فعلی (fingerprint) و دیکشنری نتایج هش‌ها، فایل مشخص‌شده را در حالت نوشتن باز کرده و ساختار کامل را با استفاده از pickle.dump سریال‌سازی و روی دیسک ذخیره می‌کند.

main _ (تابع اصلی یا بخش تحت "__main__" == __name__ if__":)

وظیفه خواندن پارامترها، بارگذاری رکوردها، بازیابی/ذخیره کش، توزیع کار روی پردازش موازی (هر تصویر توسط worker_process_image_ پردازش می‌شود)، و در نهایت نوشتن خروجی اکسل و گزارش آماری.

is_blank_image و is_xray_image برای فیلترگذاری اولیه فراخوانی می‌شوند و خود از تابع کمکی _image_entropy بهره می‌گیرند.

توابع Worker

- _worker_file_hash: محاسبه هش SHA۲۵۶ (exact_file)
- _worker_pixel_hash: محاسبه هش پیکسلی (exact_pixel)
- _worker_process_image: شامل گردش کار کامل برای یک تصویر (فیلترگذاری سفید/رادیولوژی + انتخاب نوع هش و محاسبه آن)

توابع Utility

- _json_fingerprint: محاسبه اثرانگشت ورودی JSON
- save_cache / load_cache: مدیریت کش هش‌ها
- extract_folder_id: استخراج شناسه پوشه از نام پرونده
- load_records: خواندن رکوردها از JSON
- write_excel و report_statistics: تولید خروجی‌های اکسل و آمار

جدول سرعت و زمان پردازش هر متریک:

Metric	Total time (T, s)	Speed (imgs/s)
TOTAL processing	۱۴۱ ۲۱۹.۴۱	۱۲.۵۵
hash_gpu	۱۴۱ ۰۹۱.۱۶	۱۲.۵۶
hash.xray_filter	۱۰۲ ۹۵۷.۹۲	۱۷.۴۱
hash.pixel_sha۲۵۶_gpu_wall	۳۳ ۹۹۸.۲۰	۵۲.۱۴
hash.file_sha۲۵۶	۱ ۸۴۲.۹۸	۹۶۱.۶۶

مستندات requirements :

Required:

numpy
pandas
tqdm
Pillow
openpyxl

Optional:

torch
torchvision

Standard Library:

os
json
hashlib
logging
pickle
datetime
collections
concurrent.futures
dataclasses
warnings

Artifact های پروژه :

hash_cache_all.pkl

|

├── cache_version

├── json_fingerprint

├── blank_var_thresh

├── blank_ent_thresh

├── xray_max_mean

├── xray_min_std

├── xray_max_color

├── created_at

|

├── ok

| └── image records + hashes

|

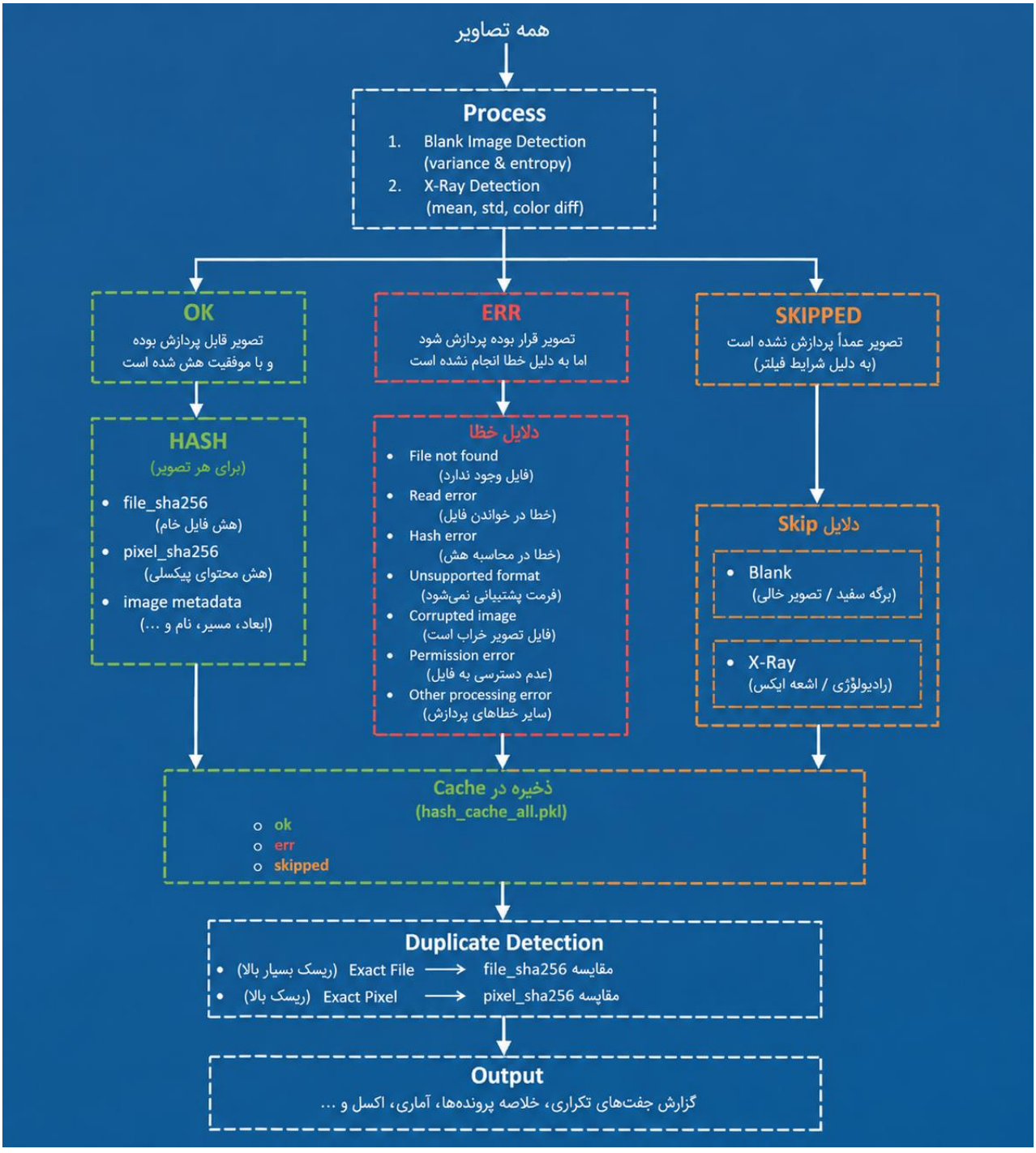
├── err

| └── failed records

|

└── skipped

    └── blank/xray records



ارتباط بین توابع:

